

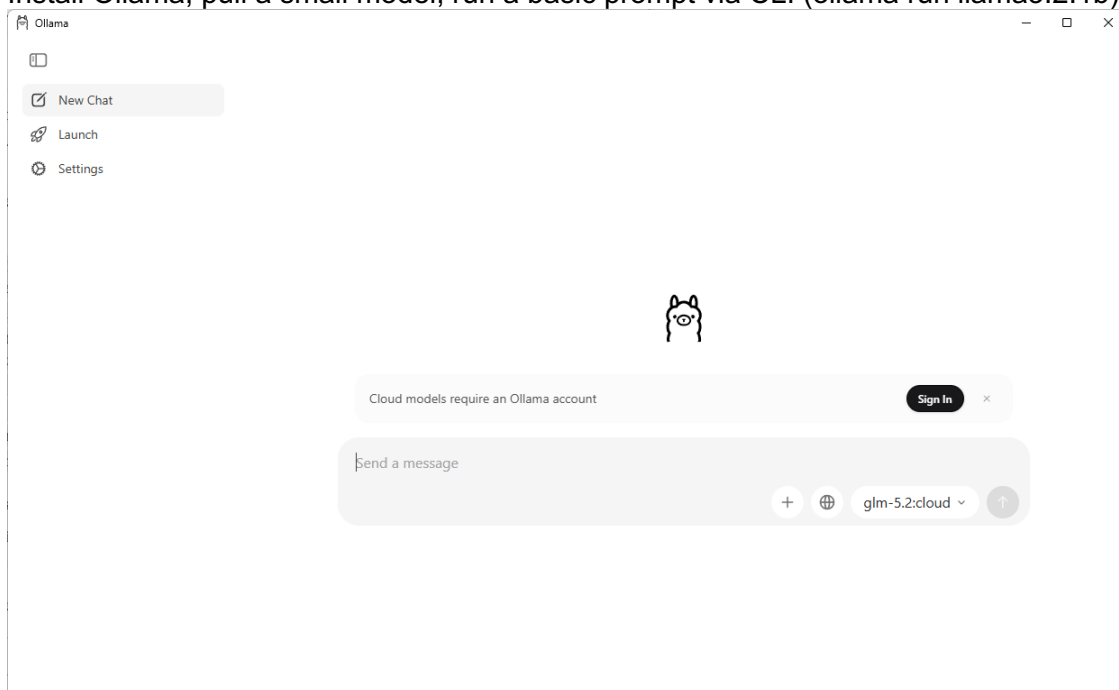
Week 1 Lab: Introduction to LLM Access (Local + Cloud)

Objective: Understand what an LLM API call actually is, then get both local and cloud access working.

Tools needed: Ollama, Jupyter Notebook

Steps:

1. Brief intro: what is an LLM, what is an API, what is Jupyter Notebook, what's the difference between calling a model locally vs over the cloud (can remove if already in lecture)
2. Install Ollama, pull a small model, run a basic prompt via CLI (`ollama run llama3.2:1b`)



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\dqduo> ollama run llama3.2:1b
pulling manifest
pulling 74701a8c35f6: 100% 1.3 GB
pulling 966de95ca8a6: 100% 1.4 KB
pulling fcc5a6bec9da: 100% 7.7 KB
pulling a70ff7e570d9: 100% 6.0 KB
pulling 4f659a1e86d7: 100% 485 B
verifying sha256 digest
writing manifest
success
>>> hello
Hello. How can I assist you today?

>>> Send a message (/? for help)

```

3. Install ollama library and call the api in notebook

```

PS D:\OTHER\IntelligentSystemsLab> pip install ollama
Collecting typing-inspection>=0.4.2 (from pydantic>=2.9->ollama)
  Downloading typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)
Downloading ollama-0.6.2-py3-none-any.whl (15 kB)
Downloading httpx-0.28.1-py3-none-any.whl (73 kB)
Downloading httpcore-1.0.9-py3-none-any.whl (78 kB)
Downloading pydantic-2.13.4-py3-none-any.whl (472 kB)
Downloading pydantic_core-2.46.4-cp313-cp313-win_amd64.whl (2.1 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 11.8 MB/s 0:00:00
Downloading annotated_types-0.7.0-py3-none-any.whl (13 kB)
Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading anyio-4.14.1-py3-none-any.whl (124 kB)
Installing collected packages: typing-inspection, pydantic-core, httpcore, anyio, annotated-types, pydantic, httpx, ollama
Successfully installed annotated-types-0.7.0 anyio-4.14.1 httpcore-1.0.9 httpx-0.28.1 ollama-0.6.2 pydantic-2.13.4 pydantic-core-2.46.4 typing-inspection-0.4.2
PS D:\OTHER\IntelligentSystemsLab>

```

```

import ollama

response = ollama.chat(
    model='llama3.2:1b',
    messages=[
        {'role': 'user', 'content': 'What is an intelligent system?'}
    ]
)

print(response['message']['content'])

```

[1] ✓ 15.5s Python

... An intelligent system is a complex computational system that can reason, learn, and interact with its environment.

Intelligent systems can be categorized into several types:

- Rule-based systems**: These systems use predefined rules to make decisions and solve problems.
- Expert systems**: These systems are based on the knowledge of a specific domain or profession and can provide advice.
- Machine learning systems**: These systems learn from data and improve their performance over time through experience.
- Neural networks**: Inspired by the human brain, neural networks use artificial neurons to process information.

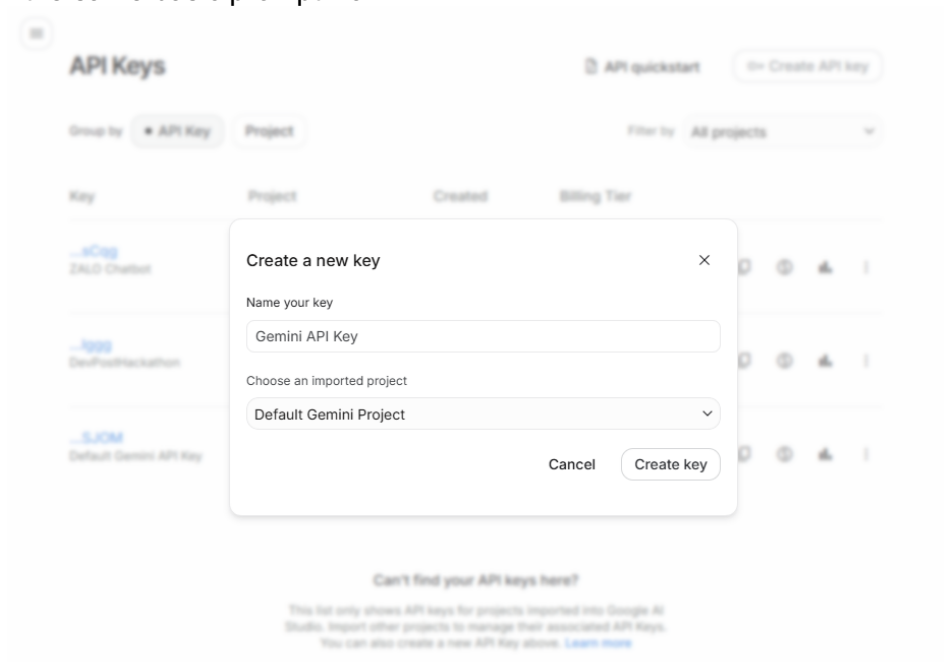
Intelligent systems possess several key characteristics:

- Cognitive abilities**: They can perceive, process, and respond to information in a way that is similar to human cognition.
- Learning**: They can learn from experience, data, or interactions with the environment.
- Reasoning**: They can reason and make decisions based on evidence and constraints.
- Autonomy**: They can operate independently without human intervention.

Intelligent systems have many applications in various fields, including:

- Robotics**: Autonomous vehicles, robots, and drones that can navigate and interact with their environment.
- Computer vision**: Systems that can understand and interpret visual data from images and videos.
- Natural language processing (NLP)**: Systems that can understand, generate, and process human language.
- Decision support systems**: Systems that provide recommendations or advice based on data analysis.

4. Explain what's happening under the hood (local server, model weights, request/response)
5. Sign up for a cloud provider (<https://aistudio.google.com/app/api-keys>), generate an API key, run the same basic prompt via API



Gemini has free tier so seems to better than OpenAI or Anthropic

```

PS D:\OTHER\IntelligentSystemsLab> pip install google-genai
984.2/984.2 kB 13.0 MB/s 0:00:00
Downloading distro-1.9.0-py3-none-any.whl (20 kB)
Downloading google_auth-2.55.2-py3-none-any.whl (256 kB)
Downloading tenacity-9.1.4-py3-none-any.whl (28 kB)
Downloading websockets-16.0-cp313-cp313-win_amd64.whl (178 kB)
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Installing collected packages: websockets, tenacity, sniffio, distro, google-auth, google-genai
  Attempting uninstall: google-auth
    Found existing installation: google-auth 2.41.1
    Uninstalling google-auth-2.41.1:
      Successfully uninstalled google-auth-2.41.1
Successfully installed distro-1.9.0 google-auth-2.55.2 google-genai-2.11.0 sniffio-1.3.1 tenacity-9.1.4 websockets-16.0

from google import genai

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="What is an intelligent system?"
)

print(response.text)

[4] ✓ 18.7s Python

... An intelligent system is a machine-based system—combining hardware and software—that can perceive its env.

Unlike traditional computer programs that follow strict, pre-written rules, an intelligent system can adapt to

Here is a detailed breakdown of how intelligent systems work, their characteristics, and real-world examples.

---

### 1. The Core Loop: Sense, Think, Act
An intelligent system operates on a continuous feedback loop consisting of three main stages:

1. Sense (Perception): The system gathers data from its environment. This is done using sensors (like cameras, microphones, or sensors in a robot).
2. Think (Reasoning & Learning): The system processes the data using artificial intelligence (AI) and machine learning algorithms to make a decision.
3. Act (Execution): The system performs an action based on its decision. This could be physical (a robot moving) or digital (a program changing a value).

```

6. Load a small provided dataset (50-100 rows)

```

import pandas as pd

df = pd.read_csv("reviews.csv")
df = df.head(20) # keep it small for the lab

```

7. Run a few simple prompts against the dataset using both local and cloud models

```

import ollama

def get_sentiment_ollama(text):
    response = ollama.chat(
        model='llama3.2:1b',
        messages=[
            {'role': 'user', 'content': f"Classify sentiment as Positive, Negative, or Neutral. Reply with one word only.\n\nText: {text}"}
        ]
    )
    return response['message']['content'].strip()

df['sentiment_ollama'] = df['text'].apply(get_sentiment_ollama)

```

```

from google import genai

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")

def get_sentiment_gemini(text):
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=f"Classify sentiment as Positive, Negative, or Neutral. Reply with one word only.\n\nText: {text}"
    )
    return response.text.strip()

df['sentiment_gemini'] = df['text'].apply(get_sentiment_gemini)

```

Compare output and speed at a high level (*Add a `time.time()` wrapper around each function to log latency per call*)

8. Ask the model to reason before answering. Compare this against a direct-answer prompt (no "think step by step") on the same question.

```

prompt = "Think step by step, then give your final answer.\n\nQuestion: A store has 3 boxes of 12 apples each. 5 apples are rotten. How many good apples remain?"

```

9. Wrap the agent in [GradioUI](#) for an interactive chat demo

```

# !pip install gradio

import ollama
import gradio as gr

def chat_with_ollama(message, history):
    response = ollama.chat(
        model='llama3.2:1b',
        messages=[{'role': 'user', 'content': message}]
    )
    return response['message']['content']

demo = gr.ChatInterface(
    fn=chat_with_ollama,
    title="Week 1: Local LLM Chat (Ollama)",
    description="Chatting with llama3.2:1b running locally via Ollama"
)

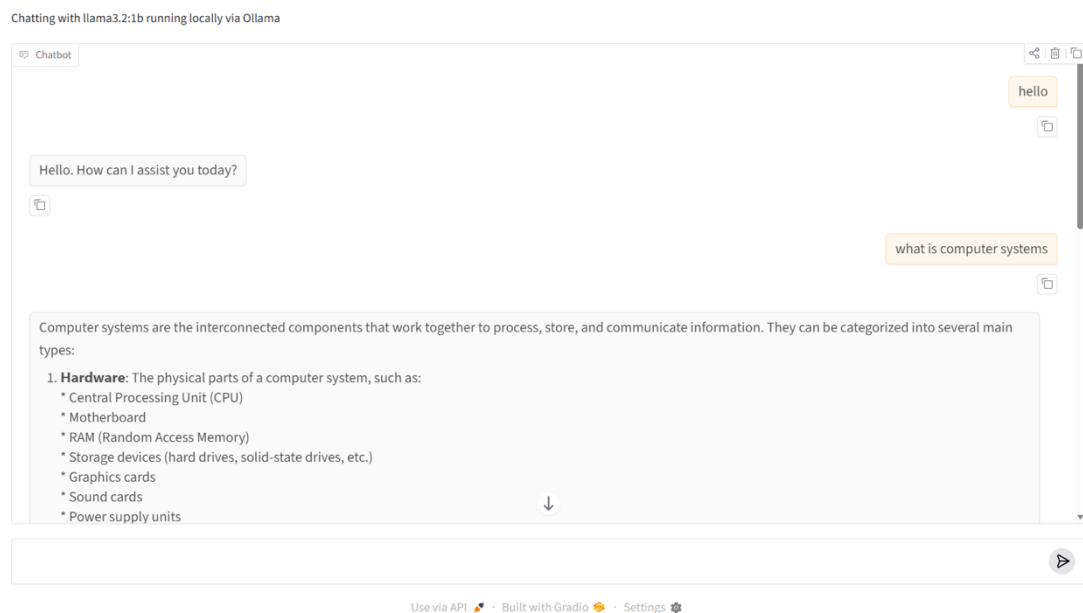
demo.launch()

```

✓ 15.6s Python

d:\OTHER\IntelligentSystemslab\.venv\lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipynb
from .autonotebook import tqdm as notebook_tqdm
* Running on local URL: <http://127.0.0.1:7860>
* To create a public link, set `share=True` in `launch()`.

Week 1: Local LLM Chat (Ollama)



Checkpoint questions:

- What's the practical difference in setup complexity between local and cloud?
- Where did the two models disagree, and why might that happen?

Deliverable: Notebook with both integrations working, plus a short comparison table (latency, cost, output quality).